

Ревью GitHub проекта

05/13/2026 10:05:48

@GitHub Проведи код ревью нашего репозитория, укажи на недостатки,...

Thinking • Addressing potential issues in application settings I'm analyzing...



@GitHub Проведи код ревью нашего репозитория, укажи на недостатки, что можно добавить, выдели слабые и сильные стороны

13.05.2026, 09:58:13



Thinking

- **Addressing potential issues in application settings**

I'm analyzing some potential issues. First, there's a settings problem with DEBUG, ALLOWED_HOSTS, and CORS settings. Then, there's the missing django-filter in requirements.txt, which could lead to a failure. Also, urls.py refers to undefined MEDIA_URL/MEDIA_ROOT, causing possible errors. I'll clarify these citations.

- **Identifying issues in password reset and frontend settings**

I noticed that the password reset feature is broken, as the route expects a URL-encoded UID, but it's actually receiving a raw numeric one. I need to address missing imports and check frontend URL settings as well. Also, there are some hardcoded values and missing imports causing errors in the frontend components. I'll make sure all of these get cited properly for reference.

- **Preparing code review with citations and detailed feedback**

I'm preparing the code review in Russian, making sure to provide thorough but balanced praise. I'll include prioritized issues, properly cite relevant parts from the README and other files, and mention the desired features, like JWT auth and CRUD functionality. I'll also make the review transparent about access limitations and ensure the citations are precise without being overly verbose. Every fact will get its citation.

✓ Thought for 2m 39s

Посмотрю репозиторий по структуре, ключевым файлам и типичным зонам риска: README, зависимости, архитектура, обработка ошибок, тесты и деплой. Потом дам ревью с сильными сторонами, минусами и конкретными улучшениями.

Пока GitHub-поиск по выбранному репозиторию не вернул файлов, поэтому проверю доступ через GitHub-коннектор другим способом. Если репозиторий не выбран в коннекторе, я честно отмечу это и дам чеклист ревью по тому, что смогу увидеть.

Нашёл README, зависимости фронтенда и бэкенда. Уже видно: стек нормальный для полноценного проекта — Django REST + React/Vite, Docker-инструкция есть; главный риск сейчас — насколько хорошо реализованы API, безопасность, тесты и продакшен-настройки.

Провёл ревью репозитория `eglayy/tatnedviga`. В целом - проект выглядит не как “пустая учебная заготовка”, а как нормальное full-stack приложение: есть Django REST backend, React/Vite frontend, JWT-аутентификация, роли пользователей, объявления, избранное, заявки на просмотр, геокодинг и Docker-инструкция. Это уже хороший объём работы, особенно если делали командой и не просто натянули шаблон.

Общее впечатление

Я бы оценил проект примерно на **7/10 как учебный/full-stack проект** и на **5/10 как production-ready приложение**.

Сильная сторона - видно, что вы не ограничились логином и списком карточек. Есть доменная логика недвижимости: собственники, клиенты, объекты, избранное, заявки на просмотр, статусы, адреса и координаты. Это хорошо видно по моделям `Property`, `Favorite`, `ViewingRequest` и кастомному пользователю с ролью `client/owner` `models.py`.

Но до “по-настоящему готового” проекта не хватает безопасности, тестов, нормальной конфигурации окружений, стабильного Docker-запуска и чуть более аккуратной архитектуры.

Сильные стороны

1. Нормальный full-stack стек

В README проект описан как приложение на **Django REST Framework + React**, с JWT, защищёнными маршрутами, CORS, SQLite и Docker-запуском `README.md`. Это хороший

выбор для проекта такого уровня: backend отдельно, frontend отдельно, API между ними.

2. Есть реальные пользовательские сценарии

Проект покрывает не только регистрацию/логин, но и основные сценарии платформы недвижимости: каталог, карточки объектов, избранное, заявки на просмотр, профиль, роли клиента и собственника. Маршруты frontend уже показывают несколько полноценных страниц: главная, профиль, избранное, карта, создание объекта, деталка объекта, логин/регистрация/сброс пароля `App.jsx`.

3. Backend-модель выглядит предметно

Модели хорошо отражают предметную область: объект недвижимости хранит цену, тип сделки, тип недвижимости, статус, изображение, адрес и координаты; избранное связано с пользователем и объектом; заявки имеют дату, время, сообщение и статус `models.py`. Это уже похоже на основу реального продукта, а не просто CRUD ради CRUD.

4. Есть попытка разделить роли и права

На backend есть проверка, что создавать объект может только пользователь с ролью `owner`, а заявки фильтруются по роли: собственник видит заявки на свои объекты, клиент - свои заявки `views.py`. Это правильное направление.

5. Хорошо, что добавили Docker

В README есть инструкции по запуску через `docker-compose`, а в репозитории есть отдельные Dockerfile для backend и frontend `README.md` `Dockerfile` `Dockerfile`. Для демонстрации проекта это большой плюс: проверяющему проще поднять приложение.

Слабые места и недостатки

1. Самая критичная проблема - настройки безопасности

В `settings.py` сейчас стоит `DEBUG = True`, `ALLOWED_HOSTS = [""]`, `CORS_ALLOW_ALL_ORIGINS = True` `settings.py`. Для разработки это допустимо, но для настоящего проекта - нет.

Что нужно сделать:

```
DEBUG = os.getenv("DEBUG", "False") == "True"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "").split(",")
CORS_ALLOWED_ORIGINS = os.getenv("CORS_ALLOWED_ORIGINS", "").split(",")
```

И отдельно завести `.env.example` с понятными переменными.

2. Есть риск, что backend не запустится из-за зависимостей

В `settings.py` используется `django_filters` в `INSTALLED_APPS` и `DEFAULT_FILTER_BACKENDS` `settings.py`, но в `backend/requirements.txt` нет пакета `django-filter` `requirements.txt`. Это может привести к ошибке импорта при запуске.

Нужно добавить:

```
django-filter
```

И лучше зафиксировать версии зависимостей, например:

```
Django==4.2.XX
django-rest-framework==...
django-filter==...
```

Сейчас зависимости без версий - это нестабильно: сегодня проект запускается, через месяц может сломаться из-за новой версии пакета.

3. Права доступа к объектам недвижимости небезопасные

В `PropertyViewSet` для `create` используется `IsOwnerUser`, но для остальных действий возвращается `AllowAny` `views.py`. Это опасно: если оставить стандартный

`ModelViewSet`, то не только просмотр, но и `update/delete` могут оказаться слишком открытыми.

При этом класс `IsOwnerOrReadOnly` уже написан, но фактически не используется `views.py`.

Лучше сделать так:

```
def get_permissions(self):
    if self.action in ["list", "retrieve"]:
        return [permissions.AllowAny()]
    if self.action == "create":
        return [IsOwnerUser()]
    return [permissions.IsAuthenticated(), IsOwnerOrReadOnly()]
```

И в `IsOwnerOrReadOnly` разрешить безопасные методы всем:

```
if request.method in permissions.SAFE_METHODS:
    return True
return obj.owner == request.user
```

4. Сброс пароля, скорее всего, сломан

В `forgot_password` в ссылку кладётся `uid = user.pk`, но в `reset_password_confirm` этот параметр декодируется через `urlsafe_base64_decode(uidb64)` `auth_views.py`. То есть backend ожидает base64 uid, а отправляет обычный id.

Нужно использовать:

```
from django.utils.http import urlsafe_base64_encode
from django.utils.encoding import force_bytes

uid = urlsafe_base64_encode(force_bytes(user.pk))
```

И уже его вставлять в reset URL.

5. Frontend слишком завязан на localhost

В `frontend/src/api.js` API URL захардкожен как `http://localhost:8000` `api.js`. То же самое повторяется в `favoriteService`, `Home`, `PropertyDetail`, `CreateProperty`, `Form` `favoriteService.js` `Home.jsx` `PropertyDetail.jsx` `CreateProperty.jsx` `Form.jsx`.

Это мешает деплою. Правильнее:

```
const apiUrl = import.meta.env.VITE_API_URL || "http://localhost:8000";
```

И все запросы делать через один общий `api` instance, а не местами через `axios`, местами через `api`.

6. Docker выглядит недоделанным

В `entrypoint.sh` есть миграции, `collectstatic` и создание суперпользователя `entrypoint.sh`, но в `backend/Dockerfile` он только копируется и получает права, а запускается обычный `python manage.py runserver` `Dockerfile`. То есть `entrypoint` фактически не используется.

Нужно заменить CMD на:

```
ENTRYPOINT ["/entrypoint.sh"]
```

или:

```
CMD ["/entrypoint.sh"]
```

Ещё момент: в `docker-compose.yml` volume `backend_db:/app/backend` может перекрыть папку Django-проекта `backend`, потому что весь `backend` монтируется в `/app`, а потом отдельный volume монтируется поверх `/app/backend` `docker-compose.yml`. Это может ломать структуру проекта. Для SQLite лучше явно хранить базу в отдельной папке, например `/app/data/db.sqlite3`.

7. Тестов фактически нет

`backend/api/tests.py` сейчас пустой - только заглушка `# Create your tests here.`
`tests.py`. Для проекта такого масштаба стоит добавить хотя бы базовые тесты:

- регистрация пользователя;
- логин и refresh token;
- создание объекта только собственником;
- запрет создания объекта клиентом;
- добавление/удаление избранного;
- создание заявки на просмотр;
- approve/reject заявки только владельцем объекта.

Это сразу поднимет проект на уровень выше.

8. Есть мелкие баги и следы спешки

В `CreateProperty.jsx` компонент использует `<Navigate />`, но импортирует только `Link` и `useNavigate` `CreateProperty.jsx`. Там же два раза подряд есть поле "Полный адрес" с одинаковым `name="address"` `CreateProperty.jsx`.

В `PropertyViewSet` два раза объявлен `perform_create`; первый вариант просто перезаписывается вторым `views.py`. Это не ломает приложение напрямую, но выглядит неаккуратно.

В `services.py` логируется начало Yandex API key: `api_key[:8]` `services.py`. Даже частично логировать ключи - плохая практика. Плюс если ключа нет, будет ошибка при `None[:8]`.

Что добавить, чтобы проект выглядел "по-настоящему"

Самые важные улучшения:

1. **Production-конфиг:** `DEBUG` из env, нормальные `ALLOWED_HOSTS`, CORS только для нужного домена, PostgreSQL вместо SQLite для деплоя.
2. **Тесты + CI:** GitHub Actions, который запускает backend tests, frontend lint и frontend build.
3. **API-документация:** Swagger/OpenAPI через `drf-spectacular` или `drf-yasg`.
4. **Демо-данные:** management command или fixture, чтобы после запуска сразу были пользователи, объекты и заявки.
5. **Нормальная обработка ошибок на frontend:** не просто `alert()` или “Ошибка при создании объекта”, а вывод конкретных ошибок API.
6. **Реальные фишки вместо демо-заглушек:** кнопка “Связаться” сейчас показывает alert/demo-сценарий `PropertyDetail.jsx`. Можно добавить контакты владельца, форму сообщения или чат.
7. **Управление объявлениями:** редактирование, удаление, деактивация, статистика просмотров. Кстати, это уже частично записано в `TODO.md`, значит команда понимает, куда двигаться дальше `TODO.md`.
8. **Скриншоты и ссылка на деплой в README:** сейчас README хороший по запуску, но для проверки проекта очень не хватает скриншотов интерфейса, демо-аккаунтов и ссылки на live demo `README.md`.

Итоговая формулировка “с похвалой, но честно”

Проект производит хорошее впечатление: видно, что команда не просто собрала шаблон, а реализовала полноценный сценарий платформы недвижимости с frontend, backend, ролями, JWT-авторизацией, объявлениями, избранным, заявками и Docker-запуском. Особенно хорошо, что есть предметная модель и попытка разделить доступ для клиентов и собственников.

Главные слабые места сейчас - безопасность backend-настроек, права доступа к объектам, отсутствие тестов, хардкод `localhost` на frontend и недоведённый Docker. Если исправить эти вещи, добавить CI, API-документацию, демо-данные и нормальный деплой, проект будет выглядеть уже не как учебная работа, а как уверенный MVP.

13.05.2026, 09:58:14